Module 9: Manage Advanced and Dynamic Inventories

4 HOURS · 25% THEORY · 75% HANDS-ON

Dynamic Architecture

Understand how dynamic inventories are structured

Cloud & CMDB

Integrate AWS, Azure, GCP, VMware, and ServiceNow

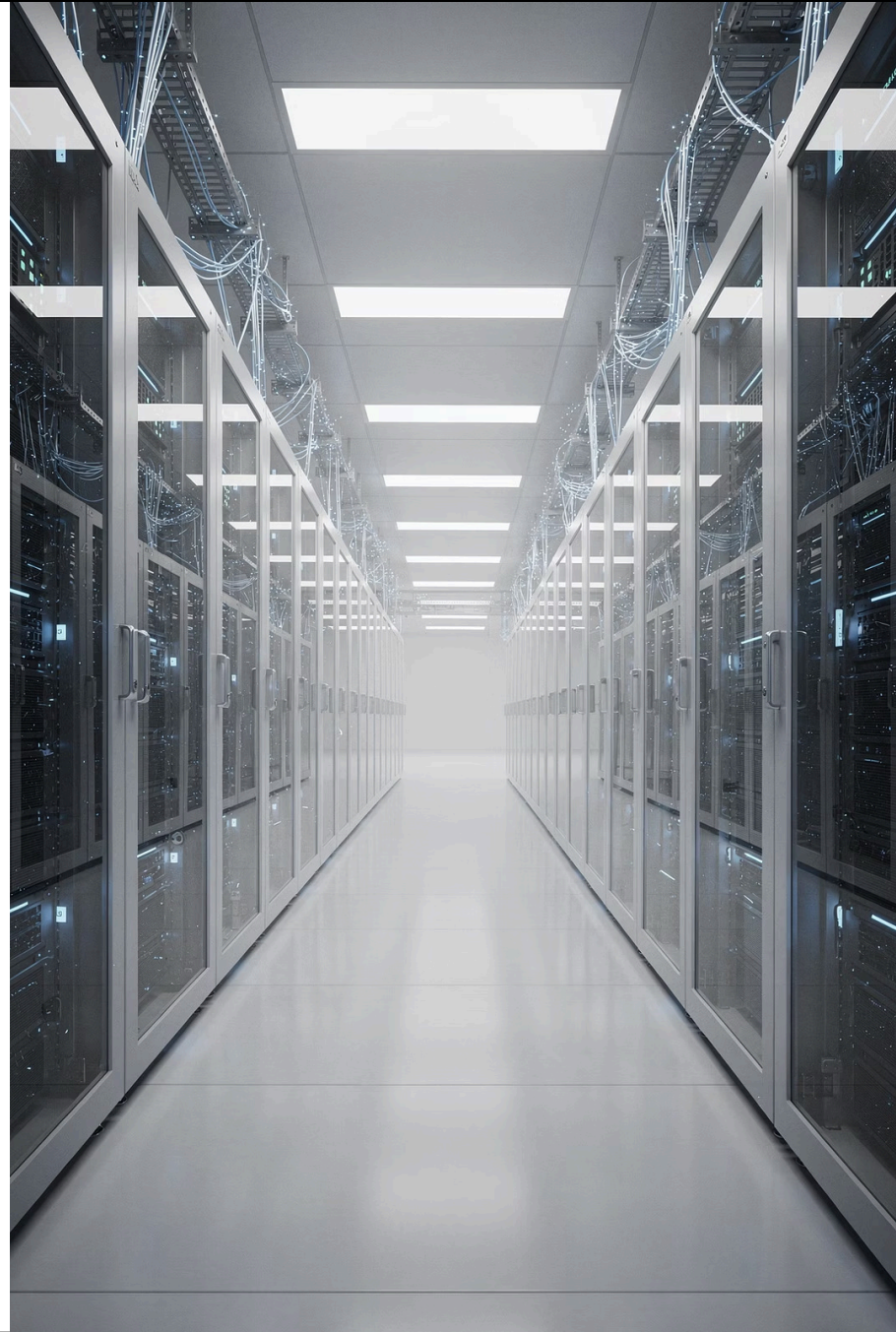
Inventory Plugins

Use and configure built-in plugins

Smart Inventories

Create and manage large-scale infrastructure

This module is **critical for enterprise automation** — static inventories do not scale.



9.1 Dynamic Inventory Concepts

Static Problem

Traditional inventories break as infrastructure changes — new servers appear, old ones are deleted, IPs shift.

```
[web]
server1
server2
server3
```

Dynamic Solution

Dynamic inventory **automatically queries external systems** to discover hosts in real time.

- AWS, Azure, GCP
- VMware, OpenStack
- CMDB, ServiceNow
- Red Hat Satellite

Real Enterprise Example

In a cloud environment, infrastructure changes constantly. Dynamic inventory keeps automation in sync automatically.

1

Auto Scaling Adds 5 Servers

Inventory updates automatically

2

DevOps Deploys New VM

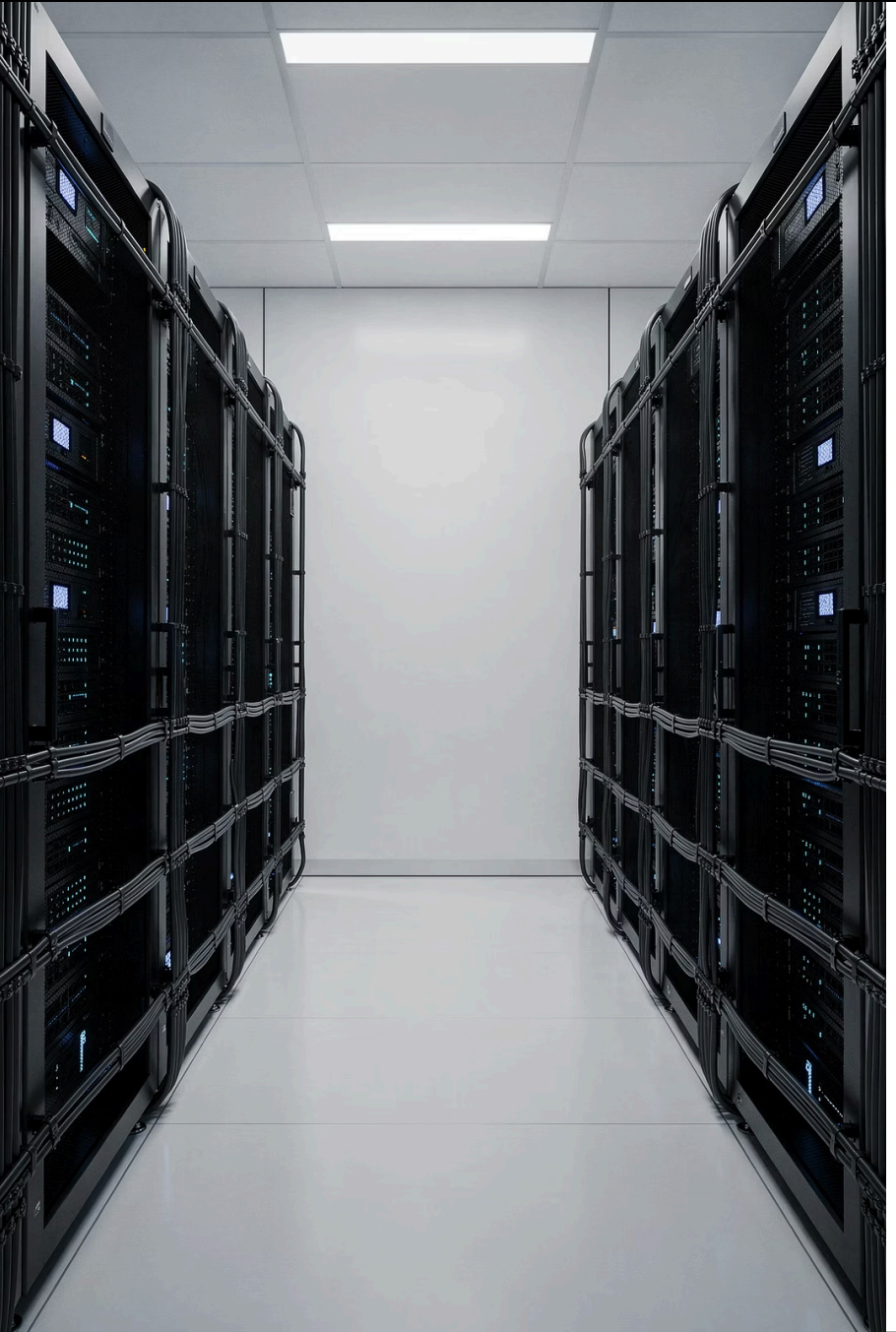
Host appears automatically

3

Server Terminated

Removed from inventory

Without dynamic inventory, automation **breaks quickly** in cloud environments.



When to Use Static Inventory

Small Labs

Fixed, predictable host lists

Fixed Infrastructure

On-prem test environments

Learning Ansible

Simple, transparent host definitions

```
[web]
server1
server2
```

When to Use Dynamic Inventory

Cloud Infrastructure

AWS EC2, Azure VMs, GCP instances

Auto-Scaling Systems

Kubernetes nodes, EC2 auto scaling

CI/CD Pipelines

VMware clusters, OpenStack, bare metal provisioning

Large Enterprise

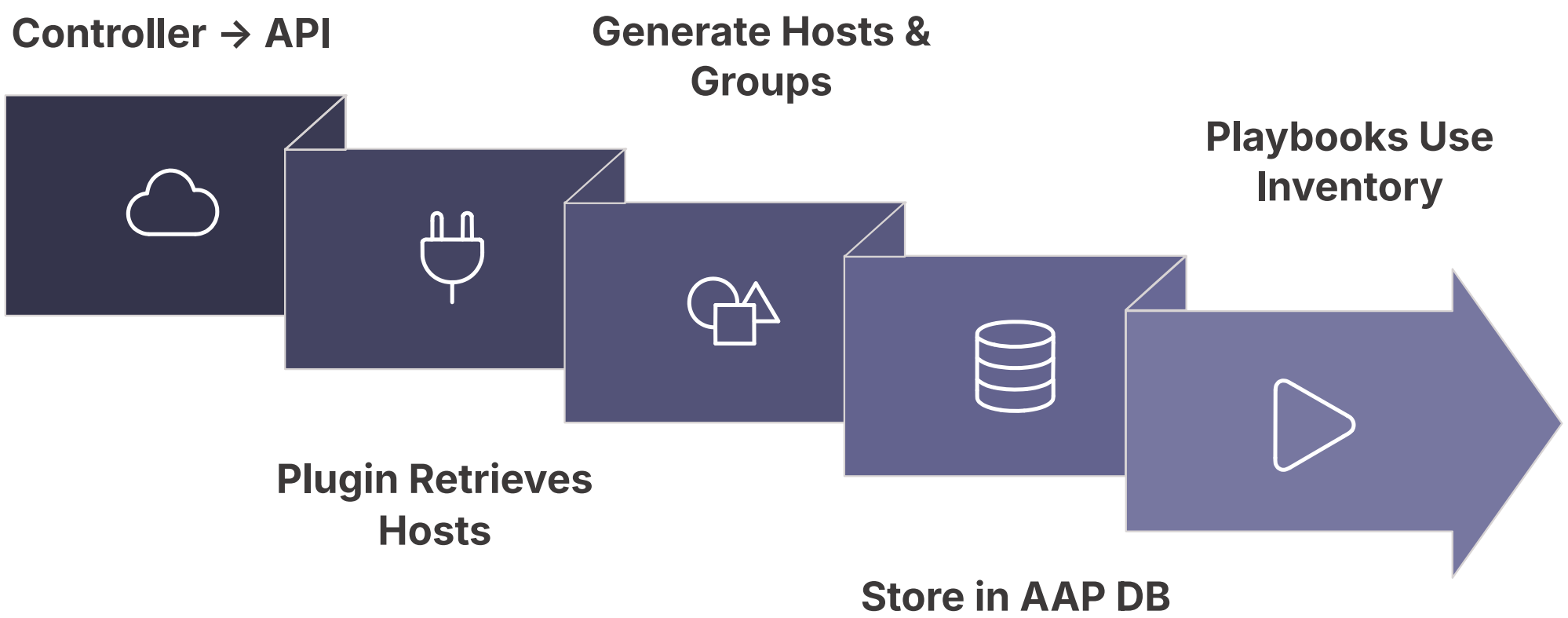
Thousands of hosts across multiple regions

Static vs. Dynamic: Comparison

Feature	Static	Dynamic
Manual updates required	Yes	No
Scales to thousands	No	Yes
Cloud integration	No	Yes
Real-time discovery	No	Yes

How Dynamic Inventories Work

Dynamic inventory operates through **inventory plugins** that bridge external platforms and AAP.



The Automation Controller handles the full lifecycle — from API query to playbook-ready inventory.

Example Workflow: AWS EC2

AAP automatically triggers the plugin, retrieves live EC2 data, and populates the inventory database — no manual steps required.

Inventory Plugins Overview

Common Plugins

Plugin	Use
aws_ec2	AWS instances
azure_rm	Azure VMs
gcp_compute	GCP instances
vmware_vm_inventory	VMware
constructed	Build groups dynamically

Example Plugin Config

```
plugin: aws_ec2
regions:
  - us-east-1
filters:
  instance-state-name: running
```

AAP automatically runs the plugin on schedule.

Cache Management Basics

The Problem

Dynamic inventory queries **external APIs** — which are expensive, rate-limited, and slow at scale.

- AWS rate limits
- Slow API response times
- Large infrastructure overhead

The Solution: Inventory Caching

Cache inventory results instead of querying every time.

```
cache_timeout = 600
```

Inventory refreshes every **10 minutes**, not on every playbook run.

Benefits of Inventory Caching

Faster Execution

No API calls on every run

Reduced API Cost

Fewer API requests = lower usage fees

Stability

Prevents hitting rate limits under load

SECTION 9.2

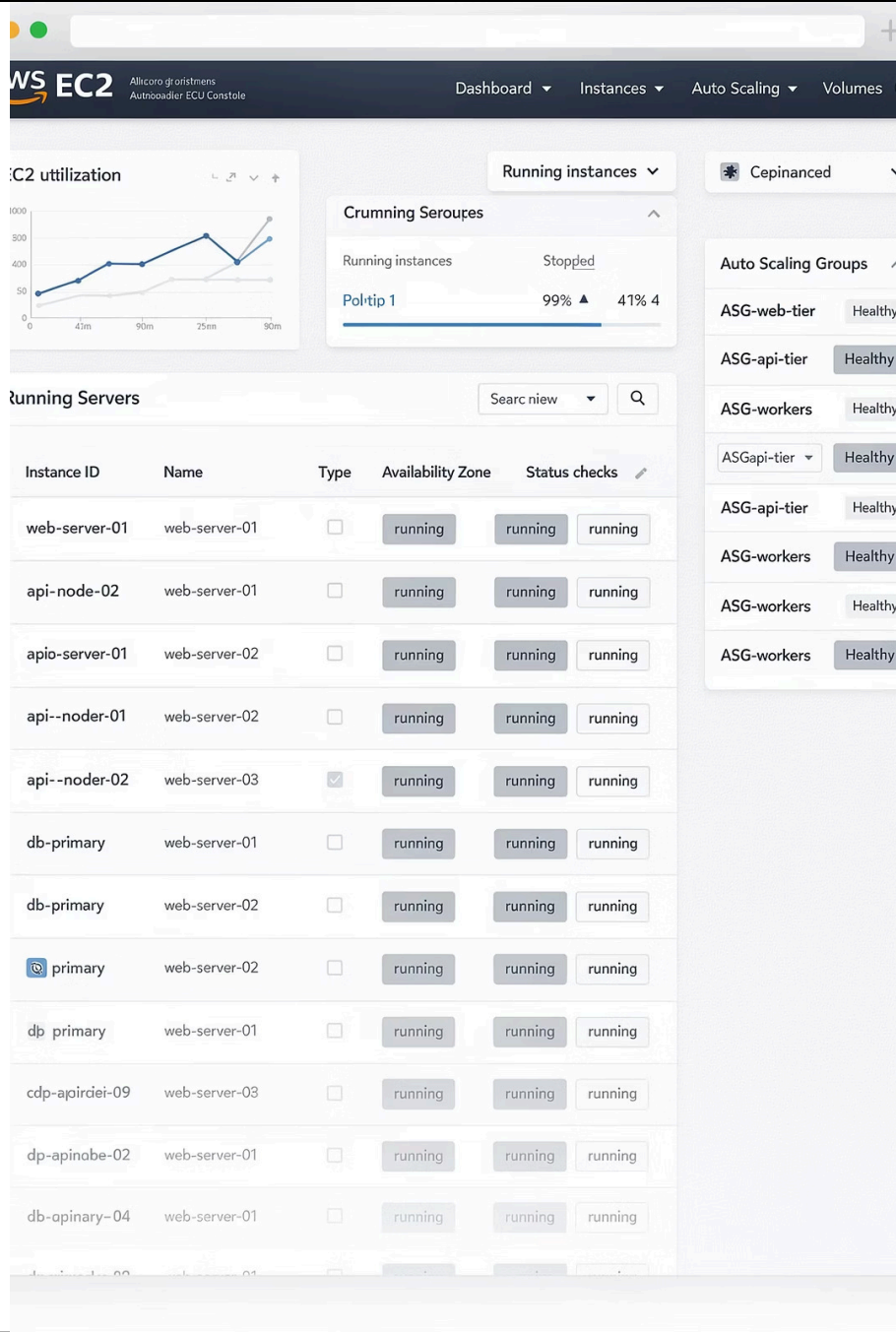
Cloud-Based Dynamic Inventories

Cloud providers are the **most common dynamic inventory sources** in enterprise Ansible Automation Platform deployments.

AWS EC2 Dynamic Inventory

One of the most widely used inventory plugins in enterprise environments.

Plugin: amazon.aws.aws_ec2



Required Credentials & Permissions

Authentication Options

IAM User

Access Key + Secret Key

IAM Role

Instance or assumed role

Env Variables

AWS_ACCESS_KEY_ID etc.

Required IAM Permissions

```
ec2:DescribeInstances
ec2:DescribeTags
ec2:DescribeRegions
```

Example IAM Policy

```
{
  "Effect": "Allow",
  "Action": [
    "ec2:DescribeInstances",
    "ec2:DescribeTags"
  ],
  "Resource": "*"
}
```

This minimal policy grants AAP read-only access to discover EC2 instances and their tags — following the principle of least privilege.

Filtering EC2 Instances

State-Based Filter

```
filters:
  instance-state-name: running
```

Only **running** instances are discovered — stopped or terminated instances are excluded.

Tag-Based Filter

```
filters:
  tag:Environment: production
```

Selects only instances tagged **Environment=production** — very common in enterprise environments.

Tag-Based Filtering

Tag-based filtering is the most common enterprise pattern for targeting specific EC2 instances.

1	2	3
Filter Config `tag:Environment: production`	**What It Selects** All instances where `Environment=production` tag is set	**Enterprise Use** Isolate prod, dev, and staging hosts without manual lists

Grouping EC2 Instances

Keyed Groups Config

```
keyed_groups:
  - key: tags.Environment
    prefix: env
```

Hosts are **automatically grouped** based on their AWS tags.

Resulting Groups

env_production

env_dev

env_stage

Common AWS Use Cases

Auto Scaling Automation

Automatically manage newly provisioned instances

Patch Management

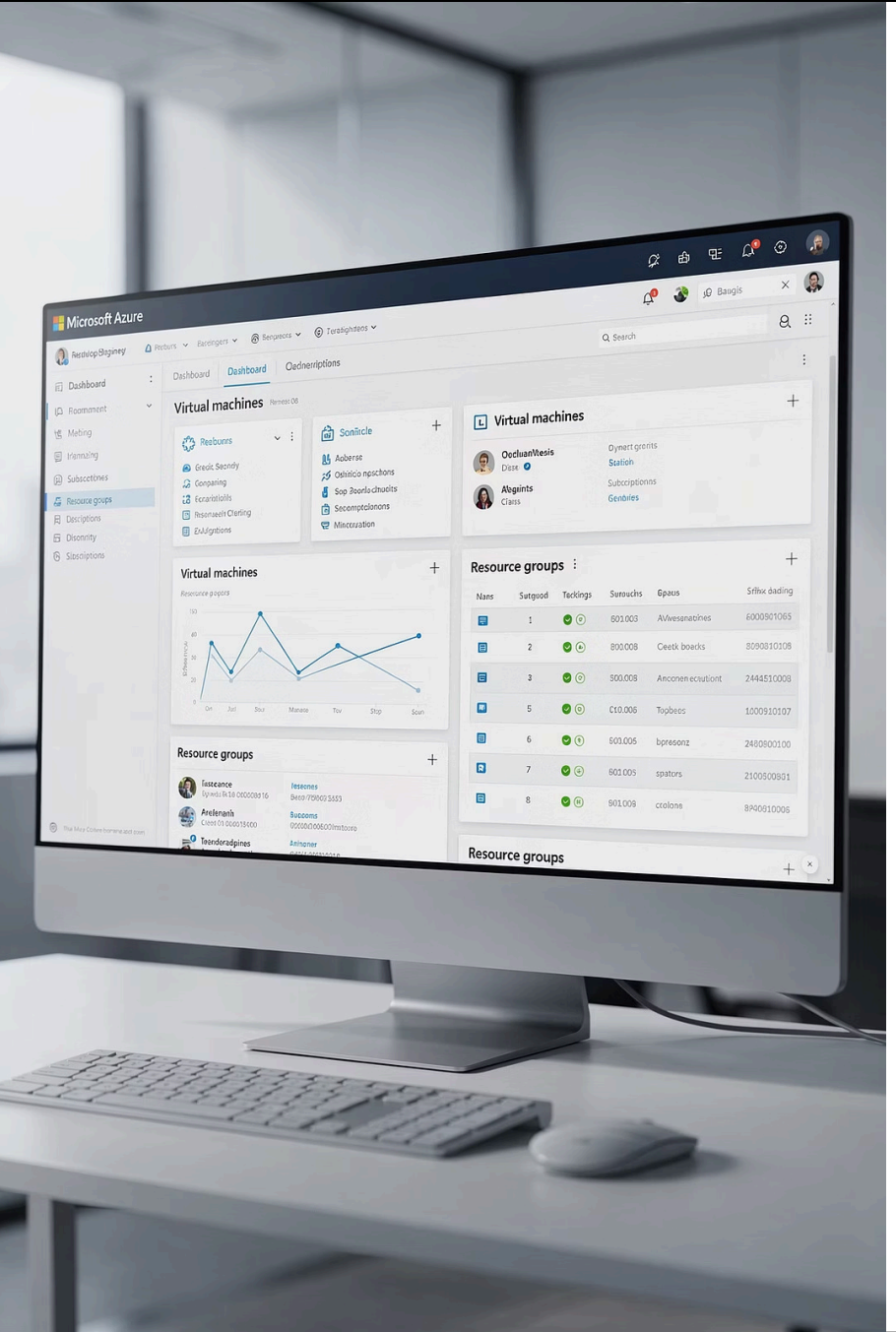
Patch all EC2 instances across regions

Backup Automation

Snapshot volumes on schedule

Security Hardening

Apply baseline configuration to all hosts



Microsoft Azure Dynamic Inventory

Plugin: azure.azcollection.azure_rm

The Azure inventory plugin integrates with Azure Resource Manager to discover and group virtual machines across subscriptions and resource groups — enabling the same dynamic automation patterns as AWS EC2.